

Submission Worksheet

Submission Data

Course: IT114-004-S2026

Assignment: IT114 Milestone 1 - 2026

Student: Stefan I. (si329)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/6/2026 11:20:19 PM

Updated: 4/6/2026 11:20:19 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-1-2026/grading/si329>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-004-S2026/it114-milestone-1-2026/view/si329>

Instructions

- Overview Link: <https://youtu.be/9dZPFwi76ak>

1. Refer to Milestone1 of any of these docs:

2. [Rock Paper Scissors](#)
3. [Basic Battleship](#)
4. [Hangman / Word guess](#)
5. [Trivia](#)
6. [Go Fish](#)
7. [Pictionary / Drawing](#)

2. Ensure you read all instructions and objectives before starting.

3. Ensure you've gone through each lesson related to this Milestone

4. Switch to the Milestone1 branch

1. `git checkout Milestone1` (ensure proper starting branch)
2. `git pull origin Milestone1` (ensure history is up to date)

5. Copy Part2 and rename the copy as Project (this new folder should be in the root of your repo)

6. Organize the files into their respective packages Client, Common, Server

1. Hint: If it's open, you can refer to the Milestone 2 Prep lesson

7. Fill out the below worksheet

1. Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
2. Since this Milestone was majorly done via lessons, the required comments should be placed in areas of analysis of the requirements in this worksheet. There shouldn't need to be any actual code changes beyond the restructure.

8. Once finished, click "Submit and Export"

9. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github

1. `git add .`
2. `git commit -m "adding PDF"`
3. `git push origin Milestone1`
4. On Github merge the pull request from Milestone1 to main

10. Upload the same PDF to Canvas

11. Sync Local

1. git checkout main
2. git pull origin main

Section #1: (2 pts.) Feature: Server Can Be Started Via Command Line And Listen To Connections

Progress: 100%

Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server started and listening
- Show the relevant snippet of the code that waits for incoming connections (before the callback to the `Server`)

```
stefan@jonovski:~/IT114/si329-it114-004-2026$ java M5.MCCS.Project.Server.Server
Server Starting
Server: Listening on port 3000
Server: Waiting for next client
```

Shows that the server is currently started

```
try (ServerSocket serverSocket = new ServerSocket(port)) {
    this.serverSocket = serverSocket; // store reference so shutdown() can close it
    while (isRunning) {
        info(message: "Waiting for next client");
        Socket incomingClient = serverSocket.accept(); // blocks until a client connects
        info(message: "Client connected");
        // third arg is a callback; ServerThread calls it once streams are open and it
        // is ready
        ServerThread serverThread = new ServerThread(incomingClient, this::onServerThread);
        serverThread.start();
        // not added to connectedClients here; that happens inside the callback after
        // setup
    }
}
```

Made obvious by the text, but `Socket incomingClient` would be the relevant line.



Saved: 4/6/2026 10:07:26 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side waits for and accepts/handles connections (before the callback to the `Server`)

Your Response:

The server has a try and catch block for successful and failed connections. The try block has while (isRunning) {, which continuously runs to listen for clients. serverSocket.accept() blocks until a connection is established. Once a connection is established, it creates a new instance of ServerThread to manage the connection between this client and the server, (serverThread only handles the connection between the server and one specific client).



Saved: 4/6/2026 10:07:26 PM

Section #2: (2 pts.) Feature: Server Should Be Able To Allow More Than One Client To Be Connected At Once

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of the server receiving multiple connections
- Show at least 3 Clients connected (best to use the split terminal feature)
- Show the relevant snippets of code that handle logic for multiple connections (the callback and the Server's knowledge of the connections)

[illegible]

Shows multiple connections from Server.java's perspective

[illegible]



Shows multiple instances of Client.java running at the same time.

```
private void start(int port) {
    this.port = port;
    info("Listening on port " + this.port);
    try {
        serversocket = new ServerSocket(port);
        this.serversocket = serversocket; // store reference so shutdown() can close it
        while (isRunning) {
            info(message: "Waiting for next client");
            Socket incomingClient = serversocket.accept(); // blocks until a client connects
            // third arg is a callback; ServerThread calls it once streams are open and it
            // is ready
            ServerThread serverThread = new ServerThread(incomingClient, this::onServerThreadInitialized);
            serverThread.start();
            // not added to connectedClients here; that happens inside the callback after
            // setup
        }
    } catch (IOException e) {
        info(message: "Error accepting connection");
        e.printStackTrace();
    } finally {
        info(message: "Server socket closed");
    }
}
```

Shows code in Server.java that makes separate threads

```
private synchronized void onServerThreadInitialized(ServerThread serverThread) {
    serverThread.setClientId(nextClientId++); // set then increase client id
    if (nextClientId < 0) { // handle overflow just in case
        nextClientId = 1;
    }
    serverThread.sendClientId(); // send the assigned client ID back to the client
    connectedClients.put(serverThread.getClientId(), serverThread);
    unicastClientStatus(serverThread); // send existing users to new client first
    // Note: broadcastClientStatus includes the new client itself (they receive their own SERVER_JOIN
    // and a SERVER_SYNC about themselves from unicast). This is intentional - filtering it out would
    // add complexity that isn't the focus of this lesson; the client handles it safely via putIfAbsent.
    broadcastClientStatus(serverThread, isJoin: true, isSync: false); // then announce new client to everyone
}
```

Shows how the server keeps track of the connected clients



Saved: 4/6/2026 10:21:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the server-side handles multiple connected clients (the callback and the Server's knowledge of the connections) and how this is achieved

Your Response:

The server-side handles connections through the use of threads. It first creates a distinct thread to handle the connection between each distinct user and the server, it uses a callback to notify the server through `onServerThreadInitialized()`, gives the client an ID, and then tracks that session using `connectedClients`.



Saved: 4/6/2026 10:21:04 PM

Section #3: (2 pts.) Feature: Client Can Be Started Via The Command Line

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the terminal output of the /name and /connect commands for each of 3 clients (best to use the split terminal feature)
- Output should show evidence of a successful connection
- Show the relevant snippets of code that handle the processes for /name, /connect, and the confirmation of being fully setup/connected



Should show everything requested above since it has /name being used, and /connect and the clients see others joining

📁 Saved: 4/6/2026 10:35:24 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the /name and /connect commands work and the code flow that leads to a successful connection for the client

Your Response:

/name: First, a user's input is piped into processClientCommand, which takes in a string. Then, it passes through each case, and in this case we are looking at /name. String name = text.replace("/name", "").trim(); creates a string called Name, and then uses the stripped text from /name to populate its value. It then passes this along to a method, User.setClientName(name). It is still however being stored locally, and it will be passed along to the server later using the

however being stored locally, and it will be passed along to the server later using the CLIENT_CONNECT payload (and after /connect is used, by sendConnectionData(myClientName)).

/connect:

First, the user's input is once again piped into processClientCommand, which takes in a string. In this case, we are looking at case Connect, where it first checks to see if the input is valid, and trims the important parts out, namely the IP address as well as the port number. It then plugs it into private boolean connect(String address, int port), which starts listening for server messages. After that, it passes sendConnectionData(myClientName) which creates a ConnectionPayload with PayloadType.CLIENT_CONNECT. On the server side, this is processed, and then it uses a setter method to set the client's given name.

 Saved: 4/6/2026 10:35:24 PM

Section #4: (2 pts.) Feature: Client Can Send Messages

Progress: 100%

≡ Task #1 (2 pts.) - Evidence

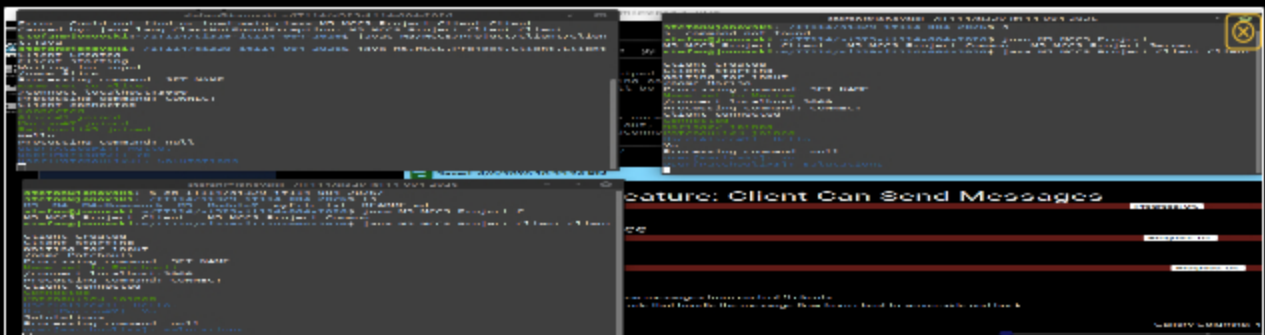
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the terminal output of a few messages from each of 3 clients
- Show the relevant snippets of code that handle the message flow from client to server-side and back



Shows the clients

```
private void sendMessage(String text) throws IOException {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.MESSAGE);
    payload.setMessage(text);
    sendToServer(payload);
}

private void sendToServer(Payload outgoingPayload) throws IOException {
    if (isConnected()) {
        out.writeObject(outgoingPayload);
        out.flush();
    } else {
        System.out.println("Not connected to server (hint: type '/connect host:port')");
    }
}
```

```

    }
    // End region for send*() methods
}

```

Shows the methods that are used to send messages from the client to the server.

```

/** Broadcasts a chat message from the sender to all clients. */
protected synchronized void handleMessage(ServerThread sender, String text) {
    broadcast(sender, text);
}

```

Server receives message from ServerThread and then calls a broadcast function (which just sends it to everyone else)

```

while (isRunning) {
    try {
        Payload fromClient = (Payload) in.readObject(); // blocks until client sends something
        if (fromClient == null) {
            throw new IOException("Connection interrupted");
        } else {
            info(textFX.colorize("Received from client: " + fromClient, color.CYAN));
            processPayload(fromClient);
        }
    } catch (ClassNotFoundException | ClassNotFoundException cce) {
        System.err.println("Error reading object as specified type: " + cce.getMessage());
        cce.printStackTrace();
    } catch (IOException e) {
        if (Thread.currentThread().isInterrupted()) {
            info(message: "Thread interrupted during read. disconnect() was called");
            break; // early exit
        }
        info(message: "IO exception while reading from client");
        e.printStackTrace();
        break;
    }
}

```

Shows BaseServerThread receiving the message from the client.

 Saved: 4/6/2026 11:20:19 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the message code flow works

Your Response:

When a client types normal text, the input is read by `listenToInput()`. If it is not a recognized command, it will use `sendMessage` to wrap it into a payload marked as `MESSAGE` which is then sent off to the server. On the serverside, `ServerThread` reads that message in `processPayload`, then passes it along to `Server.handleMessage()`, after which it is formatted, broadcasted to all other connected clients. Every client receives the `MESSAGE` payload, routes it through `processPayload()` and prints it to the terminal in `processMessage()`

 Saved: 4/6/2026 11:20:19 PM

Task #1 (1 pt.) - Evidence

Part 1:

Details:

- Show examples of clients disconnecting (server should still be active)
- Show examples of server disconnecting/closing (clients should be active but disconnected)
- Show examples of clients reconnecting when a server is brought back online
- Examples should include relevant messages of the actions occurring
- Show the relevant snippets of code that handle the client-side disconnection process
- Show the relevant snippets of code that handle the server-side termination process

```
Thread[1]: Received from client: Payload[DISCONNECT] Client id [0] Message: [null]
Thread[1]: Processing disconnect payload
Thread[1]: Sending to client: Payload[DISCONNECT] Client id [0] Message: [null]
Thread[1]: Thread being disconnected by server
Thread[1]: Closed server-side socket
Thread[2]: Sending to client: Payload[SERVER LEAVE] Client id [1] Message: [null] ClientName: [Alice]
Thread[2]: Exited thread loop. Cleaning up connection
Thread[1]: ServerThread cleanup() start
Thread[1]: ServerThread cleanup() end
Thread[4]: Received from client: Payload[DISCONNECT] Client id [0] Message: [null]
Thread[4]: Processing disconnect payload
Thread[4]: Sending to client: Payload[DISCONNECT] Client id [0] Message: [null]
Thread[4]: Thread being disconnected by server
Thread[4]: Closed server-side socket
Thread[2]: Sending to client: Payload[SERVER LEAVE] Client id [4] Message: [null] ClientName: [Patchouli]
Thread[2]: Exited thread loop. Cleaning up connection
Thread[4]: ServerThread cleanup() start
Thread[4]: ServerThread cleanup() end
Thread[2]: ServerThread cleanup() start
Thread[2]: ServerThread cleanup() end
Thread[2]: Processing disconnect payload
Thread[2]: Sending to client: Payload[DISCONNECT] Client id [0] Message: [null]
Thread[2]: Thread being disconnected by server
Thread[2]: Exited thread loop. Cleaning up connection
Thread[2]: ServerThread cleanup() start
Thread[2]: ServerThread cleanup() end
Thread[1]: ServerThread cleanup() end
```

Shows the clients disconnecting from server.java



Shows the clients disconnecting from their side

```
Thread[2]: Exited thread loop. Cleaning up connection
Thread[2]: ServerThread cleanup() start
Thread[-1]: Closed server-side socket
Thread[-1]: ServerThread cleanup() end
disconnect
```

Shows what happens after I press Ctrl+C to Server.java



Shows the clients reconnecting

```
case DISCONNECT: // request to gracefully disconnect from the server
    sendDisconnect();
```

client.java case if /disconnect is used, passes along to sendDisconnect

```
private void sendDisconnect() throws IOException {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.DISCONNECT);
    sendToServer(payload);
}
```

Send disconnect creates a new payload of type DISCONNECT

```
protected void processPayload(Payload incoming) {
    switch (incoming.getPayloadType()) {
        case CLIENT_CONNECT:
            processClientConnect(incoming);
            break;
        case DISCONNECT:
            processDisconnect(incoming);
            break;
    }
}
```

Serverthread process payload

```
private void processDisconnect(Payload incoming) {
    info(message: "Processing disconnect payload");
    Server.INSTANCE.handleDisconnect(this);
}
```

Passes to handleDisconnect

```
private void CloseServerConnection() {
    Running = false;
    CloseServerConnections();
    System.out.println("Server terminated.");
}

private void CloseServerConnection() {
    myUser.close();
    try {
        if (out != null) {
            System.out.println("Closing output stream.");
            out.close();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
    try {
        if (in != null) {
            System.out.println("Closing input stream.");
            in.close();
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
    try {
        if (server != null) {
            System.out.println("Closing connection.");
            server.close();
            System.out.println("Closed socket.");
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Client does closeServerConnection which clears knownUsers, myUser and cleans up the program.

 Saved: 4/6/2026 11:11:47 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how both client and server gracefully handle their disconnect/termination logic

Your Response:

The client sends a DISCONNECT payload when the user types /disconnect, waits for the server response and then closes its streams and its sockets. On the server side, the payload of DISCONNECT is received, which triggers a Server.handleDisconnect which removes it from the list of clients, responds back to the client and notifies the other clients using a broadcast.

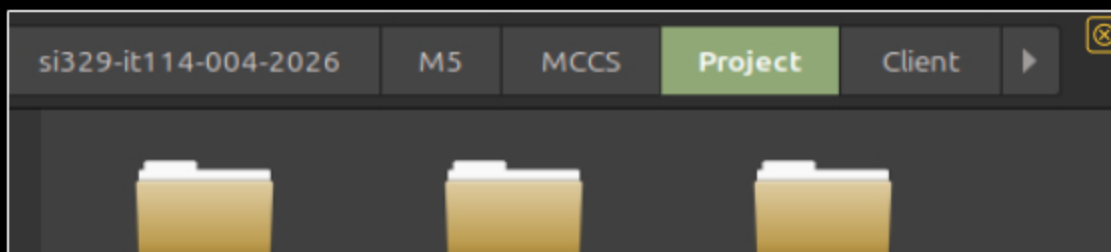
 Saved: 4/6/2026 11:11:47 PM

Section #6: (1 pt.) Misc

Progress: 100%

 Task #1 (0.25 pts.) - Show the proper workspace structure with the new Client, Common, Server, and Exceptions packages

Progress: 100%



Client

Common

Server

Shows the requested filestructure



Saved: 4/6/2026 11:12:23 PM

☰ Task #2 (0.25 pts.) - Github Details

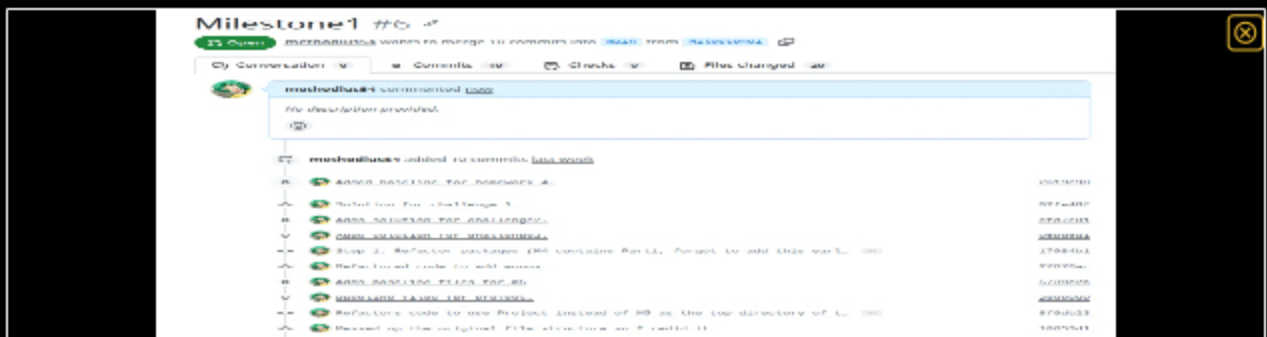
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



Shows the full edit history of Milestone1 (disregard everything but the last few)



Saved: 4/6/2026 11:16:36 PM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/methodius54/si329-it114-004-2026/pull/6>

URL

<https://github.com/methodius54/>

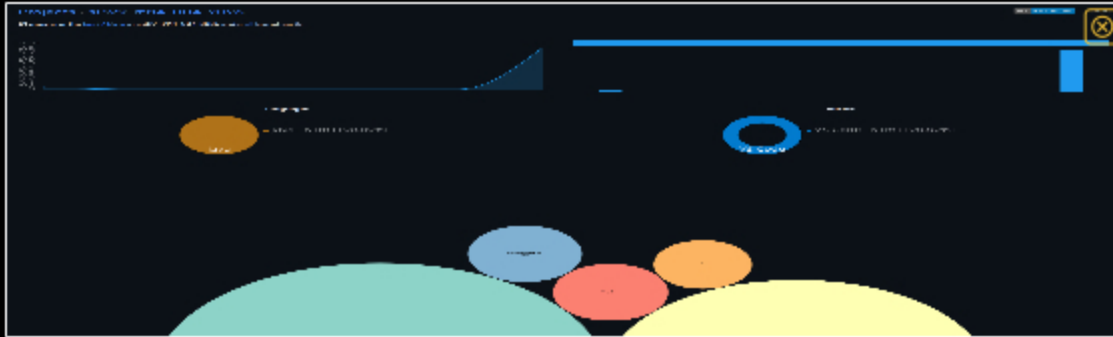
Saved: 4/6/2026 11:16:36 PM

📁 Task #3 (0.25 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Waketime screenshot



Saved: 4/6/2026 11:17:15 PM

☰ Task #4 (0.25 pts.) - Reflection

Progress: 100%

☞ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned a good chunk. Before I sat down to look at the code, some its functioning was somewhat obscure to me, but I think I understand why we are using threads, and payloads and so forth. Before doing this, I sort of just took it for granted and didn't know exactly why until I got a good look at the code's structure. Threads and singletons now make sense, we use a thread to handle each distinct connection such that several clients are not passing messages along the same upstream connection, instead we subdivide these connections so the server doesn't have conflict between what it is getting.



Saved: 4/6/2026 11:19:52 PM

☞ Task #2 (0.33 pts.) - What was the easiest part of the

assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was simply doing the connection, writing the messages and disconnecting. It was pretty simple as everything else is already set up and given to you, and there's no need to insert our own code.



Saved: 4/6/2026 11:18:38 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The screenshots and the writeup made up most of the time I used on this assignment, contributing to a good chunk of tedium, but I actually prefer doing this sort of thing where I sit down and look at the code and why and how it works rather than having to write my own from complete scratch.



Saved: 4/6/2026 11:19:09 PM